Host Entitlement / Lapsed-Payment Check

How BlueHive learns that an Off-Grid Drive subscription has lapsed, and locks itself. Two backends that never talk to each other, bridged by a signed statement carried on the device.

The idea

OGD's backend and bluehive-api have no network path between them, deliberately. Instead OGD's backend issues a signed, opaque **assertion** of account standing. OGD caches it and publishes it on the device; BlueHive reads it and relays it to its own backend, which verifies the signature with a public key and answers with a **standing** string. BlueHive cannot read or forge the token, and never tries - it acts only on the server's verdict.

#	Step	Where
1	OGD fetches a signed assertion for the account (TTL 2 days, refreshed at most every 6h).	`OGD -> its backend`
2	OGD caches it and exposes it read-only on a ContentProvider. Cache matters: an add-on opened from the TV launcher never passes through OGD.	`HostAssertionProvider`
3	BlueHive reads it - launch intent extra, then the provider, then its own mirrored copy.	`HostEntitlementRelay.submit`
4	BlueHive POSTs it to its own backend, which verifies the signature and returns a standing.	`POST /api/bluehive/ host-entitlement`
5	Only the exact string "inactive" locks the app. Everything else leaves the user alone.	`applyStanding`
6	Lock is announced on a bus and the lock screen is raised.	`EntitlementLockBus -> ServiceLockoutActivity`

Files and responsibilities

File	Responsibility
`auth/ HostEntitlementRelay.kt`	The whole client side: reads the assertion, relays it, records the verdict, owns the fail-open rule and the host-absence debounce.
`auth/EntitlementLockBus.kt`	Plain listener registry carrying "standing became inactive" from a background coroutine to an Application-level callback.
`auth/ ServiceLockoutActivity.kt`	The lock screen, with a retry that calls recheckNow().
`BlueHiveApplication.kt`	Calls init(); registers the EntitlementLockBus listener that raises the lock screen.
`auth/AuthActivity.kt`	Launch-path gate: submits any assertion the host attached, and refuses entry when already locked out.
`HomeScreenCompose.kt:234`	checkNow() on onResume - the single hook covering every return path back into the app.
`OGD entitlement/ HostAssertion.kt`	OGD side: fetches and caches the token. Sends X-Device-Fingerprint when minting.
`OGD entitlement/ HostAssertionProvider.kt`	Publishes the cached token. Cache-only by design - a provider query blocks the caller.

Parameters

Constant	Value	Meaning
`PROVIDER_URI`	`content://io.host. entitlement.provider /assertion`	Names a **capability, not a product**. Any host implementing it works; no OGD package name is hardcoded.
`STANDING_INACTIVE`	`"inactive"`	The one and only value that locks. Matched exactly, lowercased.
`SECOND_PASS_DELAY_MS`	`6000`	Corrective second read. The first read after a cold host process is answered from disk before the host has resynced, so it can be stale.
`CHECK_FLOOR_MS`	`3000`	Rate limit on checkNow(), just enough to collapse a duplicate onResume.
`RECHECK_WAKE_MS`	`4000`	How long the retry button waits for the host to wake and re-mint. Shorter than the second pass - a human is watching this one.

Constant	Value	Meaning
`ABSENT_READINGS_REQUIRED`	`3`	Readings before declaring the host gone.
`ABSENT_MIN_SPAN_MS`	`600000 (10 min)`	AND this must also elapse. Both conditions, because Android reports a package missing during updates, when storage unmounts, and when merely disabled.
`REFRESH_AFTER_MS (OGD)`	`6h, TTL 2 days`	Roughly eight refresh attempts per TTL, so missing several in a row is survivable.

SharedPreferences "host_entitlement": **assertion** (mirrored token), **standing** (last verdict, so the lock survives restarts), **host_absent_since** / **host_absent_count** (debounce state).

Key functions - HostEntitlementRelay

Function	Purpose
`submit(ctx, incoming?)`	Read the best assertion available and POST it. Sources in priority order: launch intent extra, host provider, own mirrored copy. Every failure is silent.
`submitWithCatchUp(ctx, incoming?)`	submit() plus the corrective second pass 6s later.
`checkNow(ctx)`	Home-screen hook. Rate-limited, off the main thread, and self-cancelling when the token has not changed.
`recheckNow(ctx): String?`	Suspending. Backs the retry button: reads the provider first (which also wakes the host), then reports the verdict.
`applyStanding(ctx, standing)`	Records the verdict and locks or unlocks. Fires the bus **whenever locked**, not only on transition.
`markInactiveFromServer()`	Called from the API layer on 403 entitlement_inactive. The path that survives someone clearing app data.
`isLockedOut(ctx) / lastStanding(ctx)`	Positive test against one value. Null, unknown or unrecognised must open normally.
`clearStanding(ctx)`	Forget the verdict on sign-out so the next account starts clean.
`hostPresenceReport(ctx): Boolean?`	Tri-state: true installed, false confirmed gone, **null unknown**. Null is omitted from the request entirely.
`bestAssertion(ctx, incoming?)`	The token itself, for sign-in-by-host. Shares submit()'s source ordering so the two cannot disagree.

Two rules that must not be broken

1. It fails open. Only the exact string "inactive" locks. "active", "unknown", "banned", an unrecognised value, a null body, a timeout, or no host installed at all all leave the user alone. BlueHive can be sideloaded without the host - those accounts are "unknown" forever, and locking on anything but a definite negative would shut out the entire organic install base. "I could not reach the server" and "the server says no" arrive looking similar and mean opposite things.

2. Confirmed absence only. The host-gone report marks the hardware, not the account, so it must never be a guess: three readings AND ten minutes. Everything uncertain reports null.

The server enforces the same verdict independently - bh_auth.require_user answers 403 on "inactive" - so a modified client cannot talk its way past the lock. The device is only the courier's last leg.

How it works, in plain terms

The same system, explained without assuming you write Android code.

The problem it solves

Off-Grid Drive sells the subscription. BlueHive is one of the apps that subscription pays for. When somebody's payment lapses, BlueHive has to find out — but the two services run on separate servers that, deliberately, have no connection to each other. Neither can ask the other a question. So the news has to travel some other way.

It travels with the customer

When Off-Grid Drive checks in with its own server, that server writes a short statement — this account is in good standing — and seals it with a signature only that server can produce. Off-Grid Drive keeps the sealed statement on the television and leaves it somewhere BlueHive can pick it up. BlueHive collects it and carries it to its own server, which inspects the seal and replies with a single word describing the account.

The important part is what BlueHive is *not* doing. It never opens the statement. It could not read it if it tried, and it certainly could not write one. It is a courier: it carries a sealed envelope between two offices and acts only on the reply. That is what makes the arrangement safe even though the envelope spends its life on a device the customer owns and could tamper with — there is nothing on that device worth tampering with, because the only thing that counts is a signature it cannot forge.

The statement expires on its own

Each one is valid for two days, and Off-Grid Drive quietly renews it every few hours while it is being used. This matters more than it sounds: it means nobody has to actively revoke anything. If someone stops paying, or removes Off-Grid Drive altogether, the statement simply goes stale — and a stale statement proves nothing. Access ends by expiry rather than by someone remembering to switch it off.

When the question gets asked

Every time you arrive at the home screen. That is a deliberate choice rather than a convenient one. Whatever you were doing — watching something, changing settings, switching profile, or coming back from the television's own menu — you pass through the home screen on the way. Asking in that one place therefore covers every route back into the app, with nothing else to remember to instrument. It costs almost nothing: it runs quietly in the background, never delays anything on screen, and stops immediately if the statement has not changed since last time.

Why it asks twice

There is one wrinkle worth knowing about. When the television has been off, Android clears Off-Grid Drive out of memory — so BlueHive's question is the very thing that wakes it up. Off-Grid Drive answers instantly from what it wrote down last time, before it has had any chance to check with its own server. The first answer after a cold start can therefore be out of date. BlueHive asks again six seconds later, by which point Off-Grid Drive has caught up. Almost always the second answer is identical and nothing happens. Occasionally it is the one that matters — a subscription that lapsed while the television was unplugged.

The rule that matters most: it only locks on a definite no

Exactly one reply locks the app: **inactive**. Everything else leaves the customer alone — active, unknown, an unrecognised word, a reply that never arrives, or no Off-Grid Drive installed at all.

That lopsidedness is the single most important decision in the design. BlueHive can be installed on its own, with no host app anywhere near it, and those people have never been part of a subscription — their standing reads as unknown forever. Locking on anything short of a definite refusal would shut out everybody who never owed anything in the first place. And from the app's point of view, *I could not reach the server* and *the server says no* arrive looking almost identical. Only one of them is evidence.

Put plainly: the system is built to occasionally let in somebody who should not be there, rather than ever lock out somebody who should. The first mistake costs a little revenue. The second costs a customer, a support conversation, and a good deal of trust.

Knowing whether the host app is even installed

BlueHive also reports whether Off-Grid Drive is still on the television, and that turns out to need care. Android reports an app as missing while it is mid-update, while storage is unmounted, and when it has merely been switched off rather than removed. A single glance is worth nothing. So BlueHive will only say an app is gone after several separate checks agree, spread over at least ten minutes — long enough that an update or a reconnected drive cannot be mistaken for an uninstall. This one fact describes the television rather than the account, which is precisely why it must never be a guess.

The lock does not depend on the app being honest

Finally, none of this rests on BlueHive behaving itself. Even if the app were modified to ignore the lock entirely, the server checks the same fact independently and simply refuses to serve it. The copy held on the device exists for speed and for offline use — it makes the lock instant, and it survives a restart — but it is not the thing doing the enforcing.